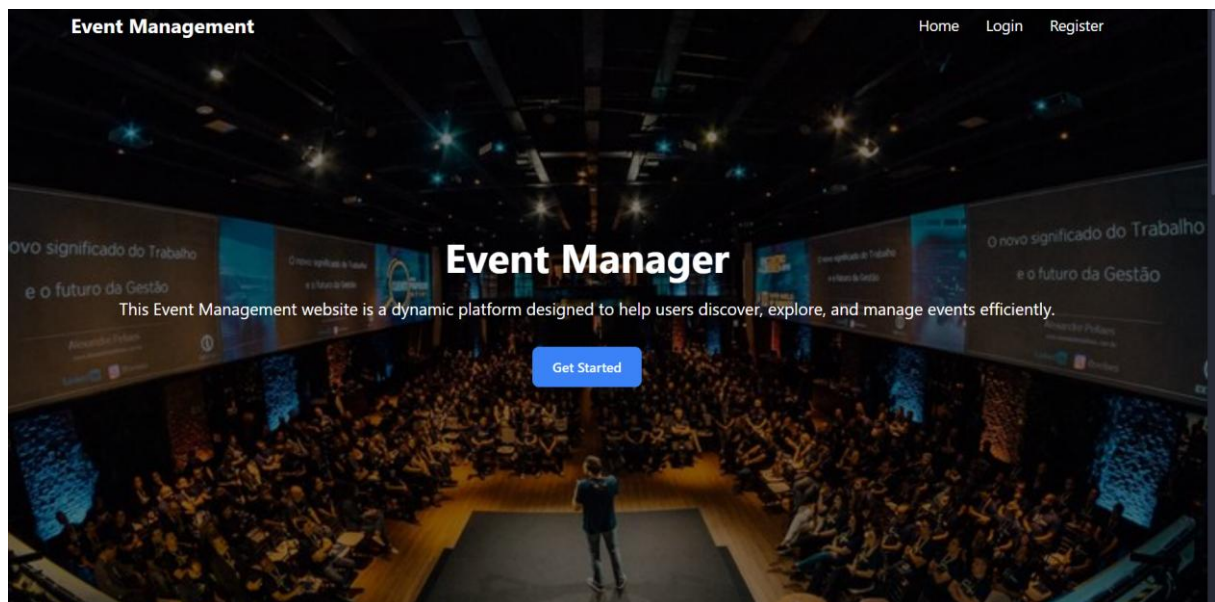


Rapport de Projet : Gestion des événements hybrides



Introduction :

Le projet Gestion des événements hybrides vise à développer une plateforme moderne et intuitive permettant la gestion d'événements hybrides, à savoir des événements accessibles à la fois en ligne et en présentiel. L'application cible principalement deux types d'utilisateurs :

- Les organisateurs, qui peuvent créer et gérer leurs événements.
- Les participants, qui peuvent explorer les événements disponibles, s'inscrire, et choisir leur mode de participation.

Cette plateforme met l'accent sur un design moderne et responsive grâce à l'utilisation de Tailwind CSS.

Objectifs du Projet :

- **Fournir un outil de gestion d'événements** : permettre aux organisateurs de créer, modifier, et supprimer des événements avec des détails complets.
- **Offrir une interface utilisateur intuitive** : permettre aux participants de consulter, rechercher et filtrer les événements disponibles.
- **Faciliter l'inscription des participants** : gestion des inscriptions avec choix du mode (en ligne ou présentiel).
- **Fournir un tableau de bord analytique** : statistiques et graphiques sur la participation pour les organisateurs.

Bibliothèques Utilisées :

Backend :

- Express.js : Framework backend.
- Mongoose : Modélisation des données MongoDB.
- Jsonwebtoken : Authentification JWT.
- Bcryptjs : Hashage des mots de passe.
- Dotsenv : Variables d'environnement.

Frontend :

- Axios : Appels API.
- React Router : Navigation.
- Recharts : Graphiques.
- Tailwind CSS : Design responsive.

Commandes pour installer les bibliothèques :

Backend :

`npm install express mongoose jsonwebtoken bcryptjs dotenv`

Frontend :

`npm install axios react-router-dom recharts tailwindcss`

Plan de Réalisation :

Étape 1 : Configuration

- Initialisation des projets backend et frontend.
- Installation des dépendances nécessaires.

Étape 2 : Développement Backend

- Création des modèles MongoDB.
- Implémentation des routes API REST.
- Tests des routes avec Postman.

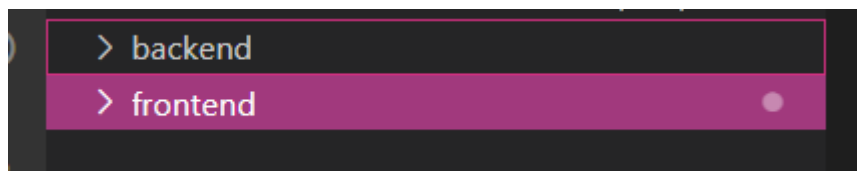
Étape 3 : Développement Frontend

- Conception des pages principales.
- Intégration des API backend.
- Application des styles avec Tailwind CSS.

Étape 4 : Tests :

- Tests API et vérification manuelle des fonctionnalités.

Architecture du Projet :



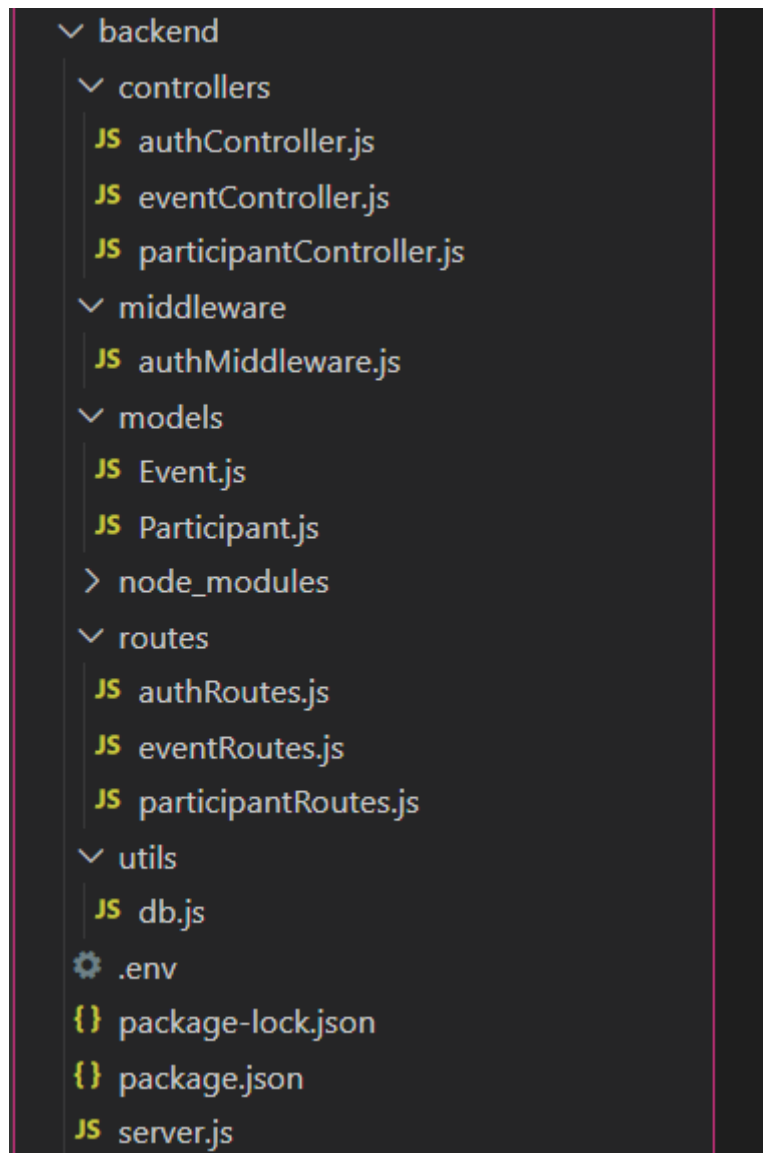
Backend :

Le backend est construit avec Express.js et utilise MongoDB pour la gestion des données.

Frontend :

Le frontend utilise React avec Tailwind CSS pour un design moderne et responsive.

Structure du backend :



Cette structure de projet représente le backend d'une application Node.js, bien organisée pour faciliter la gestion et la maintenance

controllers/ :

authController.js : Gère la logique liée à l'authentification (connexion, inscription) .

```
backend > controllers > authController.js > register > register
1  const User = require('../models/Participant');
2  const jwt = require('jsonwebtoken');
3
4  // Generate JWT token
5  const generateToken = (id, role) => {
6    return jwt.sign({ id, role }, process.env.JWT_SECRET, { expiresIn: '1h' });
7  };
8
9  // Register a new user
10 exports.register = async (req, res) => {
11   const { name, email, password, role, mode } = req.body;
12
13   // Validate required fields
14   if (!name || !email || !password || !role || !mode) {
15     return res.status(400).json({ message: 'Please fill in all fields.' });
16   }
17
18   try {
19     // Check if email already exists
20     const existingUser = await User.findOne({ email });
21     if (existingUser) {
22       return res.status(400).json({ message: 'Email already exists.' });
23     }
24
25     // Create a new user
26     const user = await User.create({ name, email, password, role, mode });
27
28     // Generate token
29     const token = generateToken(user._id, user.role);
30
31     // Respond with token and user data
32     res.status(201).json({
33       token,
34       expiresIn: 3600, // Token expiration in seconds
35       user: {
36         id: user._id,
```

- **Génération de token JWT**

La fonction generateToken crée un token JWT en utilisant l'identifiant utilisateur (id) et le rôle (role). Le token est signé avec une clé secrète définie dans process.env.JWT_SECRET et a une durée de validité d'une heure. Ce token est utilisé pour authentifier et autoriser les utilisateurs dans l'application.

- **Inscription d'un utilisateur**

L'exportation register est une fonction asynchrone qui gère l'inscription des utilisateurs. Voici les étapes principales :

- **Validation des champs requis** : Le contrôleur vérifie que tous les champs nécessaires (name, email, password, role, mode) sont fournis dans la requête.
- **Création d'un utilisateur** : Si l'email est unique, un nouvel utilisateur est créé avec les données fournies.

- **Génération du token** : Un token JWT est généré pour l'utilisateur nouvellement créé.

- **Gestion des erreurs**

Le bloc try...catch est utilisé pour capturer et gérer les erreurs :

- Si un utilisateur avec l'email fourni existe déjà, une réponse avec un code d'erreur 400 est renvoyée.
- En cas d'erreur serveur, une réponse avec un message d'erreur et un code 500 est retournée.

eventController.js : Gère les opérations liées aux événements (création, mise à jour, suppression, récupération des événements).

```
const Event = require('../models/Event');

exports.createEvent = async (req, res) => {
  try {
    const event = await Event.create(req.body);
    res.status(201).json(event);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

exports.getEvents = async (req, res) => {
  try {
    const events = await Event.find().populate('participants');
    res.status(200).json(events);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};

exports.getEventById = async (req, res) => {
  const { id } = req.params;

  try {
    const event = await Event.findById(id);
    if (!event) {
      return res.status(404).json({ message: 'Event not found.' });
    }
    res.status(200).json(event);
  } catch (err) {
    res.status(500).json({ message: 'Error retrieving event.' });
  }
};

exports.updateEvent = async (req, res) => {
  try {
    const event = await Event.findByIdAndUpdate(req.params.id, req.body, { new: true });
    res.status(200).json(event);
  } catch (err) {
    res.status(500).json({ message: 'Error updating event.' });
  }
};
```

. Création d'un événement (createEvent)

- Cette fonction asynchrone permet de créer un nouvel événement dans la base de données.

. Récupération de tous les événements (getEvents)

- Cette fonction récupère tous les événements existants depuis la base de données.
- Les participants associés à chaque événement sont également inclus grâce à la méthode `.populate('participants')`.

. Récupération d'un événement par son ID (getEventById)

- Cette fonction récupère un événement spécifique à partir de son identifiant (id) passé dans les paramètres de la requête.

. Mise à jour d'un événement (updateEvent)

- Permet de mettre à jour un événement existant en utilisant son identifiant (id) et les données fournies dans la requête.

. Suppression d'un événement (deleteEvent)

- Supprime un événement en utilisant son identifiant (id) passé dans les paramètres.

. Rejoindre un événement (joinEvent)

- Permet à un participant de rejoindre un événement.
- Vérifie si l'événement existe à partir de son identifiant (id) et si le participant n'est pas déjà inscrit (grâce à `event.participants.includes(participantId)`).

. Quitter un événement (leaveEvent)

- Permet à un participant de se retirer d'un événement.

participantController.js : S'occupe des fonctionnalités relatives aux participants.

```
1  const Participant = require('../models/Participant');
2  const Event = require('../models/Event');
3
4  exports.addParticipant = async (req, res) => {
5    try {
6      const participant = await Participant.create(req.body);
7      await Event.findByIdAndUpdate(req.body.event, { $push: { participants: participant._id } });
8      res.status(201).json(participant);
9    } catch (err) {
10     res.status(500).json({ error: err.message });
11   }
12 };
13
14 exports.getParticipantsByEvent = async (req, res) => {
15   try {
16     const participants = await Participant.find({ event: req.params.eventId });
17     res.status(200).json(participants);
18   } catch (err) {
19     res.status(500).json({ error: err.message });
20   }
21 };
22
```

Ajout d'un participant (addParticipant) : Cette fonction crée un nouveau participant en utilisant les données fournies dans la requête et met à jour l'événement correspondant en ajoutant l'ID du participant dans le tableau participants de l'événement.

Récupération des participants d'un événement (getParticipantsByEvent) : Cette fonction récupère tous les participants liés à un événement spécifique, identifié par son eventId dans les paramètres de la requête.

middleware/ :

- authMiddleware.js : Contient le middleware pour l'authentification, comme la vérification des jetons (tokens).


```

const jwt = require('jsonwebtoken');
const User = require('../models/Participant');

exports.protect = async (req, res, next) => {
  let token;

  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({ message: 'Not authorized, no token provided' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (err) {
    console.error('JWT verification failed:', err.message);
    res.status(401).json({ message: 'Not authorized, token failed' });
  }
};

exports.authorize = (roles) => {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ message: 'Not authorized, user not authenticated' });
    }

    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Not authorized to access this route' });
    }

    next();
  };
};

```

protect :

Si le token est valide, il récupère l'utilisateur correspondant à partir de la base de données (sans inclure le mot de passe) et ajoute l'utilisateur à la requête pour un usage ultérieur.

authorize :

Vérifie que l'utilisateur authentifié (défini dans protect) possède un rôle autorisé à accéder à une route spécifique.

models/ :

Event.js : Définit le schéma des événements dans la base de données.

```

const mongoose = require('mongoose');

const eventSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String, required: true },
  date: { type: Date, required: true },
  mode: { type: String, enum: ['online', 'in-person'], required: true },
  link: { type: String },
  participants: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Participant' }]
});

module.exports = mongoose.model('Event', eventSchema);

```

Participant.js : Définit le schéma des participants.

```
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema({
5    name: { type: String, required: true },
6    email: { type: String, required: true, unique: true },
7    password: { type: String, required: true },
8    role: { type: String, enum: ['organisateur', 'participant'], required: true },
9    mode: { type: String, enum: ['online', 'in-person'], required: true }, // New attribute
10  });
11
12  userSchema.pre('save', async function (next) {
13    if (!this.isModified('password')) return next();
14    this.password = await bcrypt.hash(this.password, 10);
15    next();
16  });
17
18  userSchema.methods.comparePassword = async function (enteredPassword) {
19    return await bcrypt.compare(enteredPassword, this.password);
20  };
21
22  const Participant = mongoose.model('Participant', userSchema);
23
24  module.exports = Participant;
25
```

Middleware avant sauvegarde :

- **userSchema.pre('save') :**

Cette fonction est appelée avant qu'un utilisateur soit sauvegardé dans la base de données.

Si le mot de passe n'a pas été modifié (!this.isModified('password')), la fonction continue sans hachage.

Sinon, le mot de passe est haché avec la bibliothèque bcrypt avant d'être sauvegardé, en utilisant 10 itérations pour le sel.

Méthode personnalisée : comparePassword :

Permet de comparer un mot de passe saisi (enteredPassword) avec le mot de passe haché enregistré dans la base de données.

Utilise bcrypt.compare pour cette vérification.

routes/ :

authRoutes.js : Gère les routes API pour l'authentification.

```

router.post("/register", async (req, res) => {
  const { name, email, password, role, mode } = req.body;

  try {
    const user = new Participant({ name, email, password, role, mode });
    await user.save();

    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
      expiresIn: "1h",
    });

    res.status(201).json({ message: "Utilisateur enregistré avec succès.", token });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

router.post('/login', async (req, res) => {
  const { email, password } = req.body;

  try {
    const user = await Participant.findOne({ email });
    if (!user) {
      return res.status(404).json({ message: 'Utilisateur non trouvé.' });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: 'Mot de passe incorrect.' });
    }

    const token = jwt.sign(
      { id: user._id, role: user.role },
      process.env.JWT_SECRET,
      { expiresIn: '1h' }
    );
  }
});

```

eventRoutes.js : Gère les routes API pour les événements.

```

router.post('/', protect, authorize(['organisateur']), createEvent);
router.get('/', getAllEvents);
router.get('/home', getAllEvents);
router.get('/:id', getEventById);
router.put('/:id', protect, authorize(['organisateur']), updateEvent);
router.put('/:id/join', joinEvent);
router.put('/:id/leave', leaveEvent);
router.delete('/:id', protect, authorize(['organisateur']), deleteEvent);

module.exports = router;

```

La route POST /register permet de créer un nouvel utilisateur en enregistrant ses informations (nom, email, mot de passe, rôle, mode) dans la base de données et en générant un token JWT valide pendant 1 heure.

Une réponse avec un message de succès et le token est renvoyée. La route POST /login vérifie l'identité d'un utilisateur en recherchant son email dans la base de données et en comparant son mot de passe avec celui enregistré. Si les informations sont correctes, un token JWT est généré et renvoyé.

participantRoutes.js : Gère les routes API pour les participants.

```
const express = require('express');
const router = express.Router();
const { protect, authorize } = require('../middleware/authMiddleware');
const { addParticipant, getParticipantsByEvent } = require('../controllers/participantController');

router.get('/:eventId', protect, authorize(['organisateur']), getParticipantsByEvent);

module.exports = router;
```

Route GET /:eventId : Cette route permet de récupérer les participants d'un événement spécifique, identifié par eventId. Elle utilise également les middlewares protect (authentification) et authorize (pour vérifier que seul un utilisateur ayant le rôle d'organisateur peut accéder à cette route). La fonction getParticipantsByEvent est appelée pour retourner la liste des participants.

utils/ :

db.js : Contient la logique de connexion à la base de données.

```
const mongoose = require('mongoose');
require('dotenv').config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected successfully!');
  } catch (error) {
    console.error('Error connecting to MongoDB:', error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

.env

Stocke les variables d'environnement, comme les URLs de la base de données, les clés secrètes.

```

1 MONGO_URI=mongodb://127.0.0.1:27017/eventM
2 PORT=5000
3
4

```

Fichiers de configuration :

- package.json : Contient les métadonnées du projet et les dépendances.
- package-lock.json : Verrouille les versions des dépendances pour garantir la cohérence.

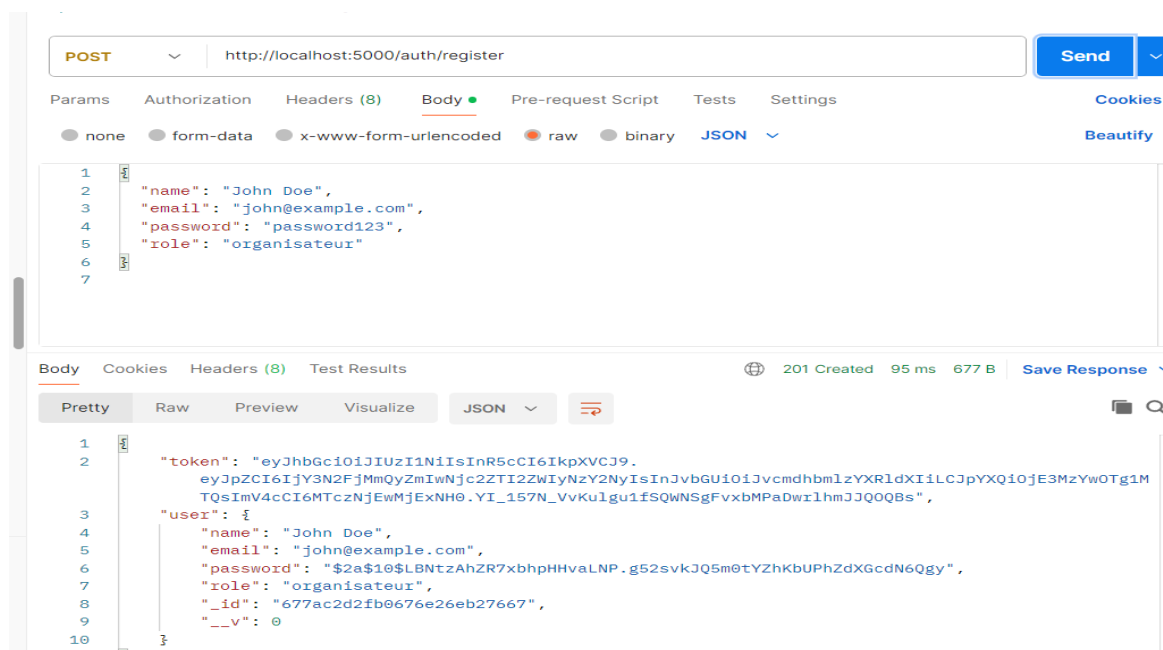
server.js :

Le point d'entrée de votre backend, qui initialise le serveur, connecte à la base de données et configure les routes.

```

backend / server.js / ...
1 // backend/server.js
2 const express = require('express');
3 const bodyParser = require('body-parser');
4 const cors = require('cors');
5 const dotenv = require('dotenv');
6 const connectDB = require('./utils/db');
7
8 const authRoutes = require('./routes/authRoutes');
9 const eventRoutes = require('./routes/eventRoutes');
10 const participantRoutes = require('./routes/participantRoutes');
11
12 dotenv.config();
13 connectDB();
14
15 const app = express();
16 app.use(cors({
17   origin: 'http://localhost:3000',
18   methods: ['GET', 'POST', 'PUT', 'DELETE'],
19   allowedHeaders: ['Content-Type', 'Authorization'],
20 }));
21 app.use(bodyParser.json());
22 app.use('/api', eventRoutes);
23 app.use('/auth', authRoutes);
24 app.use('/events', eventRoutes);
25 app.use('/participants', participantRoutes);
26
27
28 app.use((err, req, res, next) => {
29   console.error(err.stack);
30   res.status(500).json({ message: 'Internal Server Error', error: err.message });
31 });
32
33 const PORT = process.env.PORT || 5000;
34 app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
35

```



Test du login :

The screenshot displays a REST client interface with a POST request to `http://localhost:5000/auth/login`. The request body is a JSON object containing email and password. The response is a 200 OK status with a JSON body containing a token and user details.

Request:

```
POST http://localhost:5000/auth/login
{
  "email": "john@example.com",
  "password": "password123"
}
```

Response:

```
200 OK 66 ms 672 B
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY3N2FjMmQyZmIwNjc2ZTI2ZWYyNyIsInZjbGUiOiJvcmdhbmIzYXRldXIiLCJpYXQiOiJlY3MzYwOTg1NzcsImV4cCI6MTczNjEwMjE3N30.mqaIDGYsMVvgX7-gTNubAuZUpyj77vW0MTMNonVmBFY",
  "user": {
    "_id": "677ac2d2fb0676e26eb27667",
    "name": "John Doe",
    "email": "john@example.com",
    "password": "$2a$10$LBntzAhZR7xbpHHvaLNP.g52svkJQ5m0tYZhKbUPhZdXGcdN6Qgy",
    "role": "organisateur",
    "__v": 0
  }
}
```

Ajout d'un évènement avec la vérification du token :

http://localhost:5000/events

POST http://localhost:5000/events

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Conférence 2026 ",
3   "description": "Une conférence sur la science .",
4   "date": "2024-02-01T10:00:00Z",
5   "mode": "en ligne",
6   "lien": "https://conference-tech2024.com"
7 }
8
```

Body Cookies Headers (8) Test Results 401 Unauthorized 8 ms 319 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Not authorized, token failed"
3 }
```

Récupération de tout les événements :

http://localhost:5000/events

GET http://localhost:5000/events/

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies Beautify

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Conférence ",
3   "description": "Une conférence .",
4   "date": "2024-02-01T10:00:00Z",
5   "mode": "en ligne",
6   "lien": "https://conference-tech2024.com"
7 }
8
```

Body Cookies Headers (8) Test Results 200 OK 10 ms 630 B Save Response

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "_id": "677ac55b3371a0be8f26ef92",
4     "name": "Conférence 2025 ",
5     "description": "Une conférence sur la science .",
6     "date": "2024-02-01T10:00:00.000Z",
7     "mode": "en ligne",
8     "participants": [],
9     "__v": 0
10  },
11  {
12    "_id": "677ac5c63371a0be8f26ef9b",
13    "name": "Conférence ",
14  }
15 ]
```

Modification d'un évènement :

The screenshot shows a REST client interface with a PUT request to `http://localhost:5000/events/677ac5003371a0be8f26ef8f`. The request body is a JSON object representing an event. The response is a 200 OK status with a response time of 10 ms and a body size of 442 B. The response body is a JSON object containing the updated event details, including an `_id` and an empty `participants` array.

```
PUT http://localhost:5000/events/677ac5003371a0be8f26ef8f

{"name": "Conférence 2025 ", "description": "Une conférence .", "date": "2024-02-01T10:00:00Z", "mode": "en ligne", "lien": "https://conference-tech2024.com"}
```

Body Cookies Headers (8) Test Results 200 OK 10 ms 442 B Save Response

```
1 {
2   "_id": "677ac5003371a0be8f26ef8f",
3   "name": "Conférence 2025 ",
4   "description": "Une conférence .",
5   "date": "2024-02-01T10:00:00.000Z",
6   "mode": "en ligne",
7   "participants": [],
8   "__v": 0
9 }
```

Suppression d'un évènement :

The screenshot shows a REST client interface with a DELETE request to `http://localhost:5000/events/677ac5003371a0be8f26ef8f`. The response is a 200 OK status with a response time of 12 ms and a body size of 294 B. The response body is a JSON object containing a `message` field with the value "Event deleted".

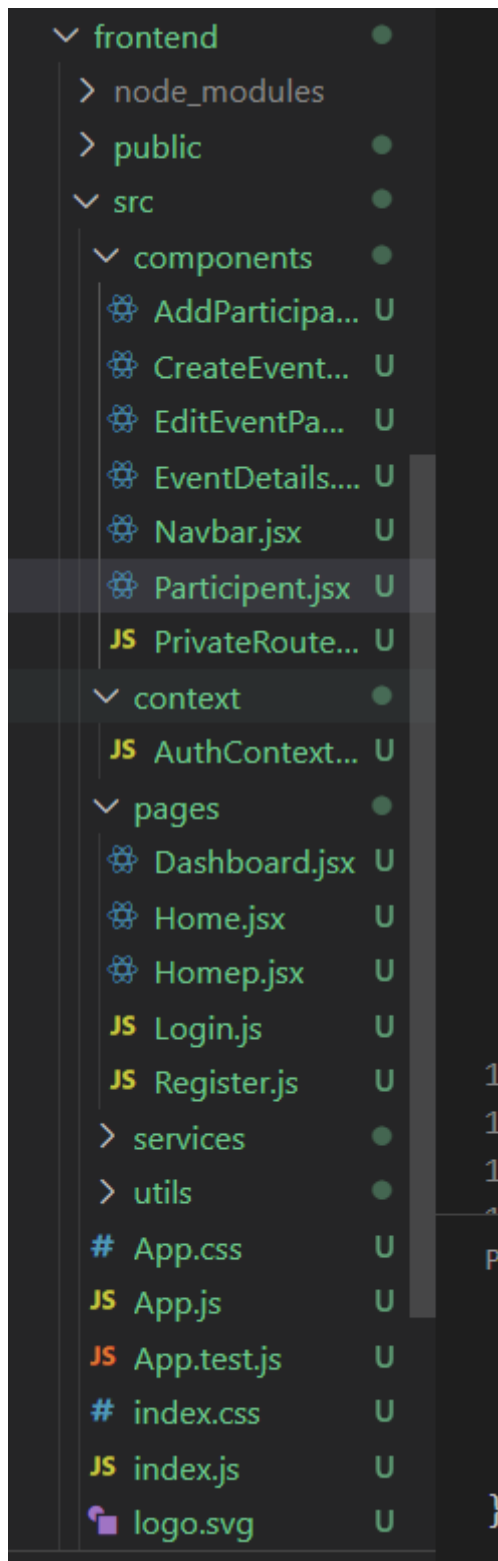
```
DELETE http://localhost:5000/events/677ac5003371a0be8f26ef8f

{"name": "Conférence 2025 ", "description": "Une conférence .", "date": "2024-02-01T10:00:00Z", "mode": "en ligne", "lien": "https://conference-tech2024.com"}
```

Body Cookies Headers (8) Test Results 200 OK 12 ms 294 B Save Response

```
1 {
2   "message": "Event deleted"
3 }
```

Structure du Frontend:



components/ :

Contient des composants réutilisables pour l'application, tels que :

- AddParticipant.jsx, CreateEvent.jsx, et EditEventParticipant.jsx : Composants pour gérer les événements et les participants.
- EventDetails.jsx : Affiche les détails d'un événement spécifique.
- Navbar.jsx : Barre de navigation.
- Participant.jsx : Gère l'affichage ou les interactions liées aux participants.
- PrivateRoute.jsx : Composant pour protéger certaines routes en vérifiant l'authentification.

context/ :

- AuthContext.js : Fournit un contexte pour gérer l'état global de l'authentification (par exemple, les tokens JWT et les utilisateurs connectés).

pages/ :

- Contient les pages principales de l'application :
 - Dashboard.jsx : Page du tableau de bord, probablement pour les organisateurs.
 - Home.jsx et Homep.jsx : Pages d'accueil ou principales pour différents rôles/utilisateurs.
 - Login.js et Register.js : Pages pour la connexion et l'inscription.

services/ :

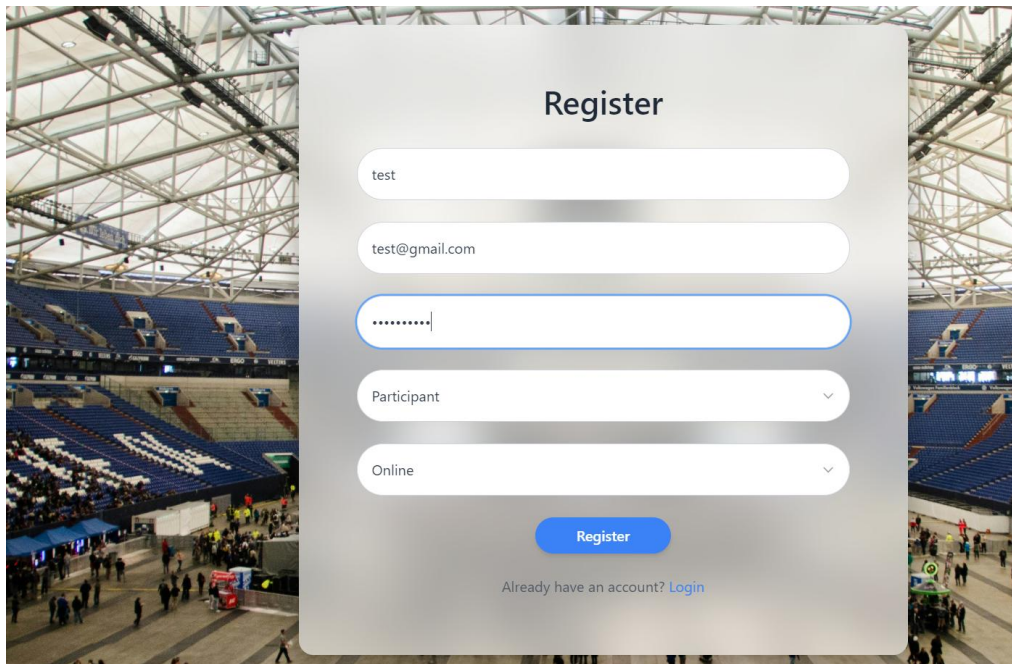
- (Non affiché dans les détails, mais généralement utilisé pour gérer les appels API).

Fichiers principaux :

- App.js : Point d'entrée principal pour définir les routes et le rendu des composants.
- App.css et index.css : Styles globaux de l'application.
- index.js : Point de départ de l'application React, où tout est monté sur le DOM.
- logo.svg : Logo utilisé dans l'application.

Frontend test :

Test de la page d'inscription :



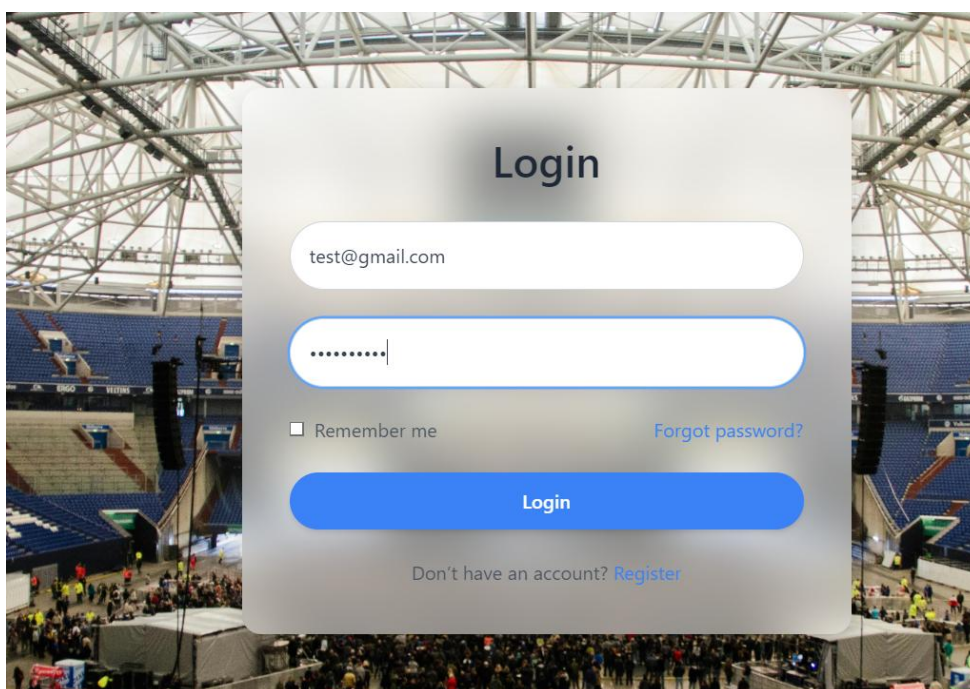
The image shows a 'Register' form overlay on a background of a large stadium with blue seats and a white structural roof. The form is a light gray rectangle with rounded corners. It contains the following elements:

- Title:** 'Register' in bold black text at the top center.
- Inputs:** Five white input fields with rounded ends. The first three contain 'test', 'test@gmail.com', and a masked password '.....'. The last two are dropdown menus with 'Participant' and 'Online' selected.
- Buttons:** A blue 'Register' button at the bottom center.
- Link:** A blue 'Login' link below the button with the text 'Already have an account?'.

Vérification de la création du compte dans la base de donnée :

```
_id: ObjectId('67785df809d59ef82e7ba7bb')
name: "test"
email: "test@gmail.com"
password: "$2a$10$Ii8EcnNKWiB/mvGXGyCs.vvNbfsZr6Ltl1qZnoXNNSGm576.Vnu6"
role: "organisateur"
mode: "in-person"
__v: 0
```

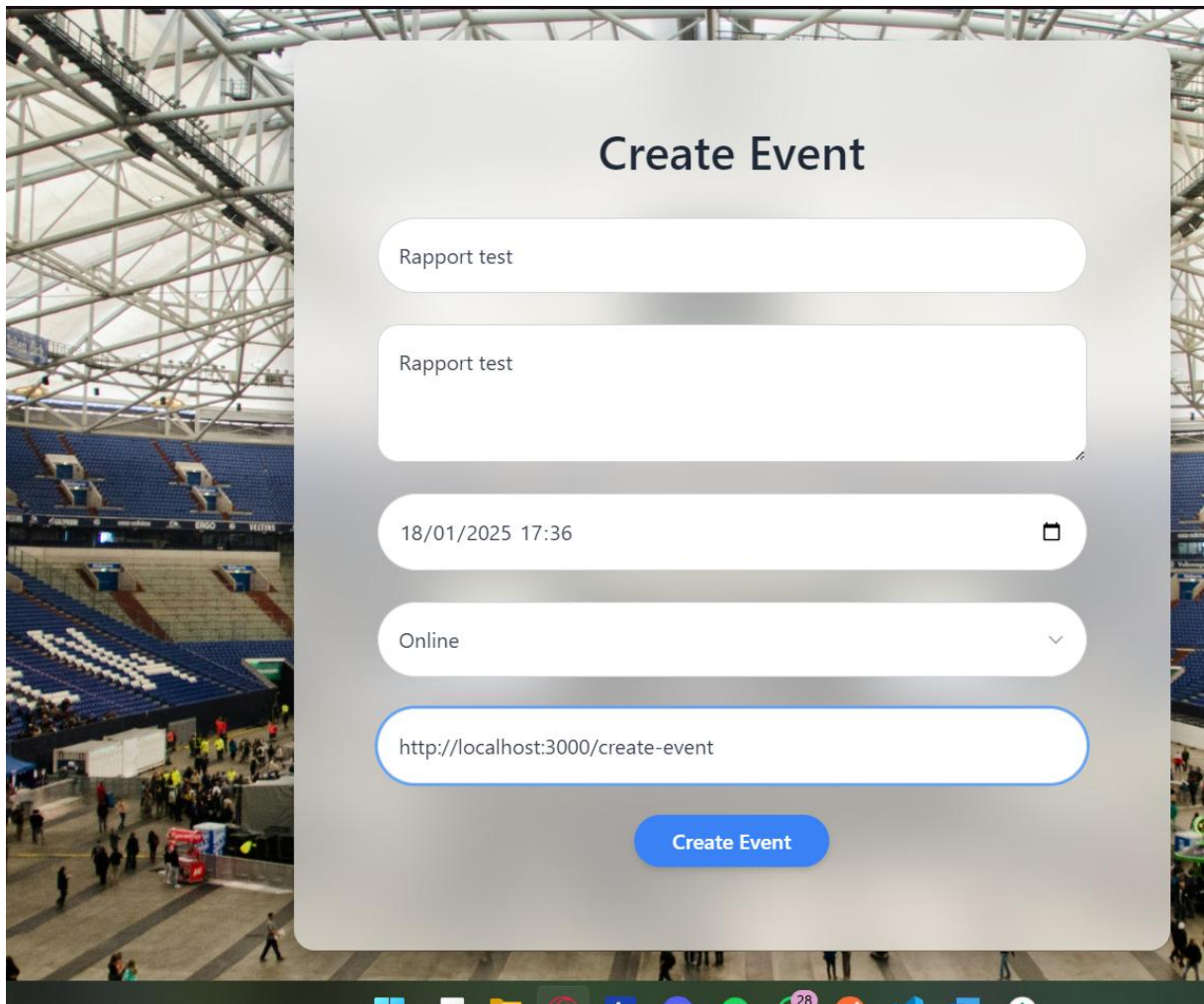
Test de la page de login:



The image shows a 'Login' form overlay on the same stadium background as the register form. The form is a light gray rectangle with rounded corners. It contains the following elements:

- Title:** 'Login' in bold black text at the top center.
- Inputs:** Two white input fields with rounded ends. The first contains 'test@gmail.com' and the second contains a masked password '.....'.
- Form Elements:** A 'Remember me' checkbox with the text 'Remember me' and a blue 'Forgot password?' link to its right.
- Buttons:** A blue 'Login' button at the bottom center.
- Link:** A blue 'Register' link below the button with the text 'Don't have an account?'.

Test de la page de création des évènements :



The screenshot shows a 'Create Event' form with the following fields and values:

- Name:** Rapport test
- Description:** Rapport test
- Date:** 18/01/2025 17:36
- Mode:** Online
- Link:** http://localhost:3000/create-event

A 'Create Event' button is located at the bottom of the form.

Vérification de la création du compte dans la base de donnée :

```
_id: ObjectId('677ab530c6d431b70946554b')
name: "Rapport test"
description: "Rapport test"
date: 2025-01-18T16:36:00.000+00:00
mode: "online"
link: "http://localhost:3000/create-event"
participants: Array (empty)
__v: 0
```

Déploiement Avec Docker :

Fichier Dockerfile Backend.

```
2 FROM node:18
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm install
6 COPY . .
7 EXPOSE 5000
8 CMD ["node", "server.js"]
9
```

Fichier Dockerfile Frontend.

```
frontend > Dockerfile
1 # Use Node.js image to build the application
2 FROM node:18 AS build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm install
6 COPY . .
7 RUN npm run build
8
9 # Use Nginx to serve the built files
10 FROM nginx:alpine
11 COPY --from=build /app/build /usr/share/nginx/html
12 EXPOSE 80
13 CMD ["nginx", "-g", "daemon off;"]
14
```

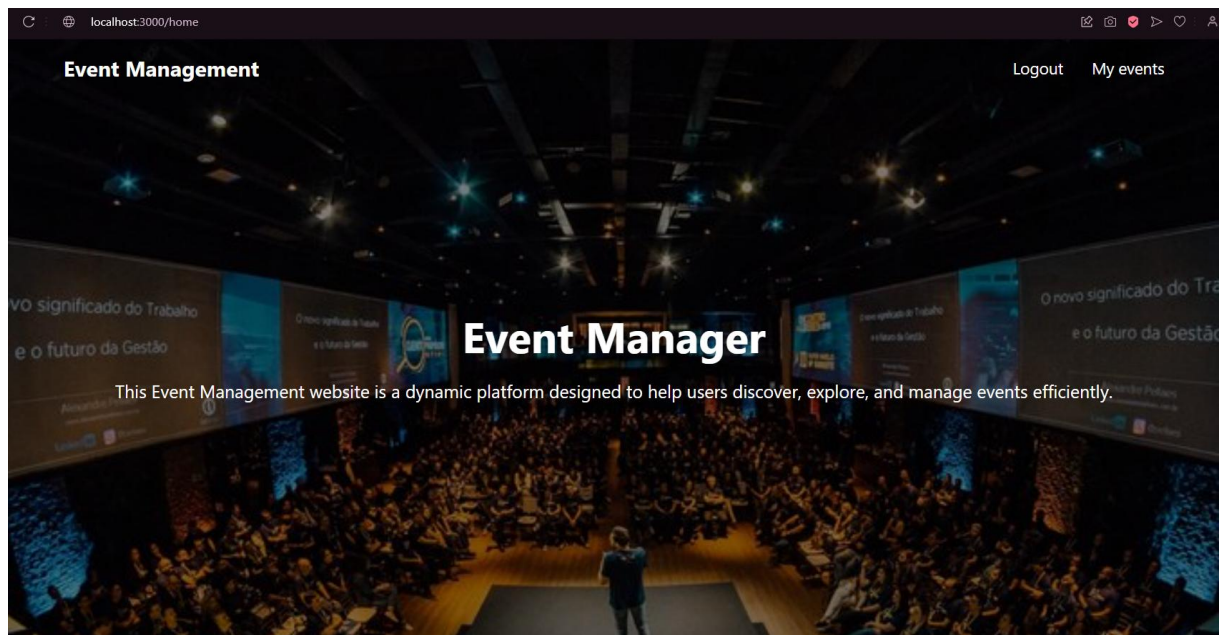
Fichier docker-compose.yml

```
docker-compose.yml
3  services:
4    frontend:
5      build:
6        context: ./frontend
7        dockerfile: Dockerfile
8      ports:
9        - "3000:80" # Map port 80 in the container to port 3000 on the host
10     depends_on:
11       - backend
12
13     backend:
14       build:
15         context: ./backend
16         dockerfile: Dockerfile
17       ports:
18         - "5000:5000" # Map port 5000 in the container to port 5000 on the host
19       environment:
20         MONGO_URL: mongodb://mongo:27017/event # MongoDB URL
21       depends_on:
22         - mongo
23
24     mongo:
25       image: mongo
26       ports:
27         - "27017:27017" # Map port 27017 in the container to port 27017 on the host
28       volumes:
29         - mongo-data:/data/db
30
```

Création des containers :

```
PS C:\Users\lenovo\Downloads\Event Manager3\Event Manager3> docker-compose build
time="2025-01-06T21:23:49+01:00" level=warning msg="C:\\Users\\lenovo\\Downloads\\Event Manager3\\Event Manager3\\docker-compose.yml: the att
be ignored, please remove it to avoid potential confusion"
[+] Building 3.1s (23/23) FINISHED
=> [backend internal] load build definition from Dockerfile
=> => transferring dockerfile: 196B
=> [frontend internal] load metadata for docker.io/library/node:18
=> [backend internal] load .dockerignore
=> => transferring context: 2B
=> [frontend build 1/6] FROM docker.io/library/node:18@sha256:7f31a1eb14c61719b8bb0eaa029310cc33851f71d3578cc422b390f8096977c5
=> [backend internal] load build context
=> => transferring context: 144.46kB
=> CACHED [frontend build 2/6] WORKDIR /app
=> CACHED [backend 3/5] COPY package*.json ./
=> CACHED [backend 4/5] RUN npm install
=> CACHED [backend 5/5] COPY . .
=> [backend] exporting to image
=> => exporting layers
=> => writing image sha256:11cb3d2623d3cef7ee65bfac19b1dd8e30df942fb79d90ba0dfc4f0b20ec52c7
=> => naming to docker.io/library/eventmanager3-backend
=> [backend] resolving provenance for metadata file
=> [frontend internal] load build definition from Dockerfile
=> => transferring dockerfile: 349B
=> [frontend internal] load metadata for docker.io/library/nginx:alpine
=> [frontend internal] load .dockerignore
=> => transferring context: 2B
=> [frontend stage-1 1/2] FROM docker.io/library/nginx:alpine@sha256:41523187cf7d7a2f2677a80609d9caa14388bf5c1fbca9c410ba3de602aaaaab4
=> [frontend internal] load build context
=> => transferring context: 3.47MB
=> CACHED [frontend build 3/6] COPY package*.json ./
=> CACHED [frontend build 4/6] RUN npm install
```

Lancement des containers :



Conclusion du Projet

Ce projet de gestion des événements hybrides est une solution complète et bien structurée, combinant un backend robuste et un frontend dynamique. Grâce à l'utilisation de technologies modernes comme **Node.js**, **Express**, **MongoDB** pour le backend et **React** pour le frontend, l'application offre des fonctionnalités essentielles telles que la gestion des utilisateurs (authentification et autorisation), la gestion des événements (création, mise à jour, suppression, participation), et une interface utilisateur intuitive et réactive.

Le backend suit une architecture claire avec des contrôleurs, des middlewares, et des modèles bien définis, garantissant une séparation des préoccupations et une sécurité renforcée grâce à l'utilisation des tokens JWT. Le frontend, organisé autour de composants réutilisables et d'une gestion centralisée de l'état (via un contexte d'authentification), permet une expérience utilisateur fluide et efficace.